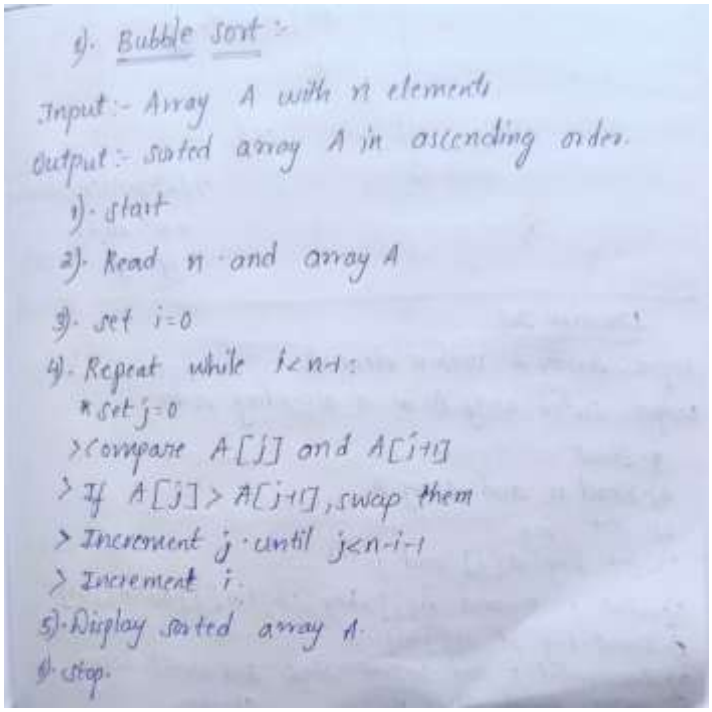
PRACTICAL - 1

AIM : Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

BUBBLE SORT ALGORITHM



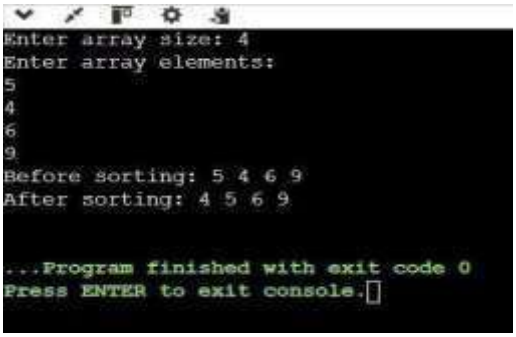
PROGRAM:

```
#include <iostream>
using namespace std; class
bubblesort { public: int
size, arr[100], i; void
arrayinput()

…. { cout << "Enter array size: ";
cin >> size; cout << "Enter array
elements:\n"; for (i = 0; i < size;
i++)
{ cin
>> arr[i];
}
```

1

```
    }      void printarray()
    {            for (i = 0; i <
size; i++)
        {               cout <<
arr[i] << " ";
        }         cout
<< endl;         }
void bubblelogic()
{         int j, temp;
for
(i = 0; i < size - 1; i++)
        {             for (j = 0; j < size - i - 1;
j++)
            {                 if
(arr[j] > arr[j + 1])
{                   temp =
arr[j];                     arr[j]
= arr[j + 1];
arr[j
+ 1] = temp;
                }
            }
        }
}         };        int       main()
{       bubblesort                    bs;
bs.arrayinput();     cout << "Before
sorting: ";       bs.printarray();
bs.bubblelogic();     cout <<
"After  sorting:           ";
bs.printarray();     return 0;
}
```

OUTPUT :


Enter array size: 4
Enter array elements:
5
4
6
9
Before sorting: 5 4 6 9
After sorting: 4 5 6 9
...Program finished with exit code 0
Press ENTER to exit console.

TIME COMPLEXITY : Step Count / Frequency Count Method)

Bubble sort Time complexity :

```
for(i=0; i<n-1; i++) {
    for(j=0; j<n-i-1; j++) {
        if( arr[j] > arr[j+1] {
        temp = arr[i]
        arr[j] = arr[j+1]
        arr[j+1] = temp
    }
}
```

Best case : $\Omega(n)$
worst case : $O(n^2)$
Avg case : $O(n^2)$

$T(n) = (n-1)(n-1)+4$
$= n^2-2n+4$
$T(n) = O(n^2)$

(i). Best case :- (The array is sorted)

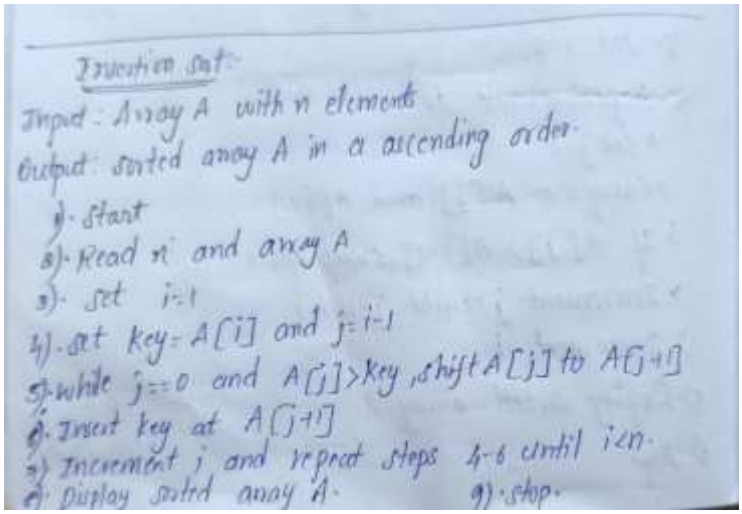
no of comparision : $0-1$
$T(n) = n-1$
$T(n) = O(n)$

(ii). Average case :- $(n-1)+(n-2)+(n-3)+\ldots+2+1$
$1+2+3+\ldots(n-1)$
$= n(n-1)/2$
$T(n) = n(n-1)/2 + (n^2-n)$
$T(n) = O(n^2)$

(iii). Wurst case (The array is in reverse order)

eg:- 50 40 30 20 10
$(n-1)+(n-2)+(n-3)+\ldots+2+1$
$1\times2\times3\ldots(n-1)$
$= n(n-1)/2 \Rightarrow T(n) = (n^2-n)/2$
$T(n) = O(n^2)$

INSERTION ALGORITHM



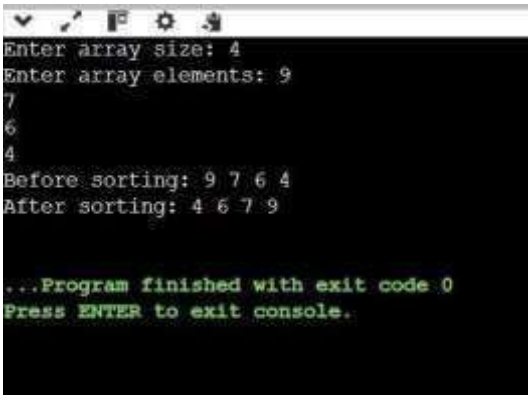
PROGRAM:

```
#include <iostream>  using
namespace std;

class InsertionSort  { public:      int
size, arr[100], i, j, temp;

   void arrayinput()
   {          cout << "Enter array
size: ";          cin >> size;
cout << "Enter array elements: ";
for (i = 0; i < size; i++)
                                                                  {                cin
>> arr[i];
       }
   }

   void printarray()
```

```
   {              for (i = 0; i
< size;
i++)
{             cout << arr[i] <<
" ";
      }         cout
<< endl;
   }

   void insertionsortlogic()
{         for (i = 1; i < size; i++)
      {             temp
= arr[i];             j =
i - 1;

          while (j >= 0 && arr[j] > temp)
          {                 arr[j +
1] = arr[j];                 j--;
          }

          arr[j + 1] = temp;


      }
} };

int main()
{
   InsertionSort is;

   is.arrayinput();


   cout << "Before sorting: ";
is.printarray();

   is.insertionsortlogic();
```

```
    cout << "After sorting: ";
is.printarray();

return 0;

}
```

OUTPUT :



TIME COMPLEXITY : Step Count / Frequency Count Method)

```
for(i=1; i<n; i++)        ——— n
{
  key = arr[i];           ——— 1
  j = i-1;                ——— 1
  while (j>=0 && arr[j]>key) — 1
  {
    arr[j+1] = arr[j];    ——— 1
    j--;
  }
  arr[j+1] = key;         ——— 1
}
T(n) = O(n)
```

Best case: O(n)
Avg case: $O(n^2)$
Worst case: $O(n^2)$

Average case:-

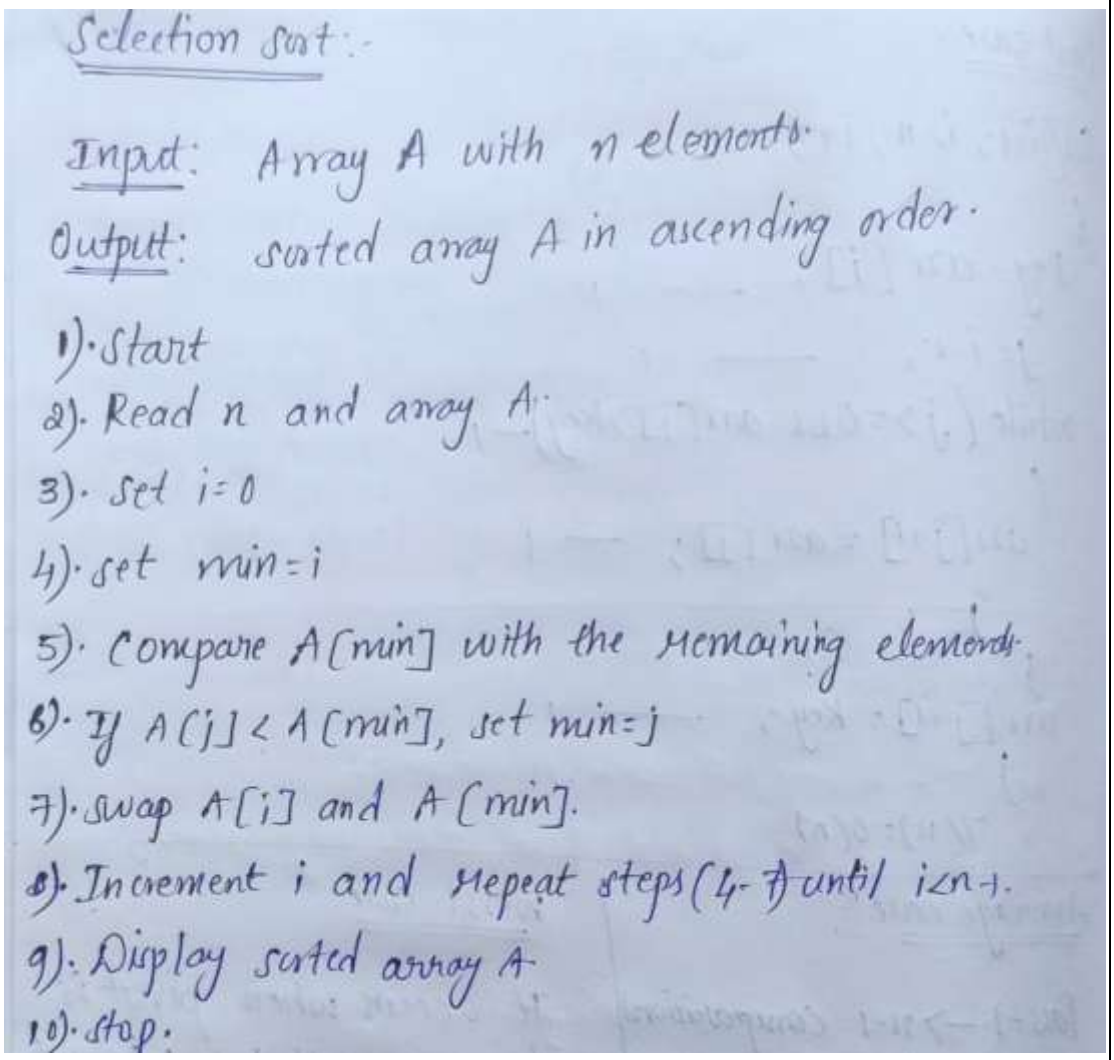
Pass-1 → n-1 comparisions
Pass-2 → n-2 comparisions
$\therefore T(n) = \frac{n(n-1)}{2}$
$= O(n^2)$

Worst case:

It occurs when pivot is always smallest (or) largest.
$n+(n-1)+(n-2)+\ldots+1$
$T(n) = \frac{n(n+1)}{2}$
$= O(n^2)$

Best case: Array sorted

$n+1$
$T(n) = n-1$
$T(n) = O(n)$



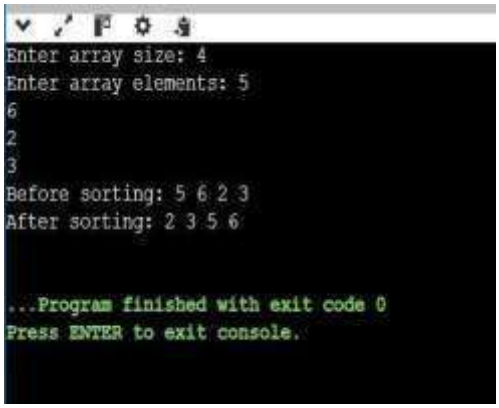
PROGRAM:

```
#include <iostream> using
namespace std;

class SelectionSort  { public:
int size, arr[100], i, j, temp;
void arrayinput()
   {          cout << "Enter array
size: ";          cin >> size;
```

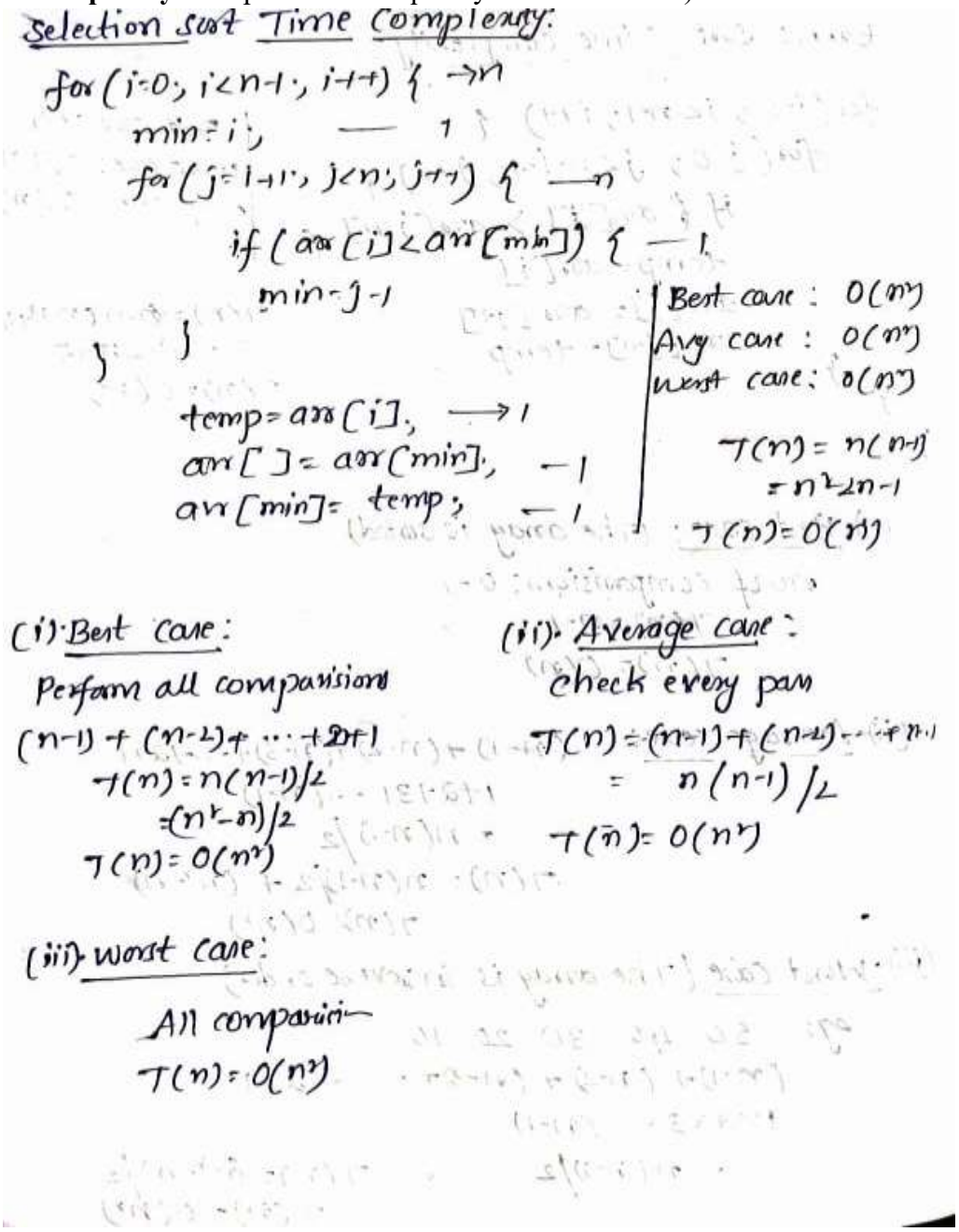
```
cout << "Enter array elements: ";
for (i = 0; i < size; i++)
        {               cin
>> arr[i];
        }
    }

    void printarray()
    {           for (i = 0; i < size; i++)
        {               cout << arr[i]
<< " ";
        }           cout
<< endl;
    }

    void selectionsortlogic()
    {           for (i = 0; i < size - 1;
i++)
        {               int min
= i;

            for (j = i + 1; j < size; j++)
            {                   if (arr[j]
< arr[min])                     {
min = j;
                }
```

```
        }               if
(min != i)
{               temp = arr[i];
arr[i] = arr[min];
arr[min]  = temp;
        }
      }
   }
};

int main()
{
   SelectionSort ss;

   ss.arrayinput();

   cout << "Before sorting: ";     ss.printarray();
ss.selectionsortlogic();

   cout << "After sorting: ";
ss.printarray();

return 0;
}
```

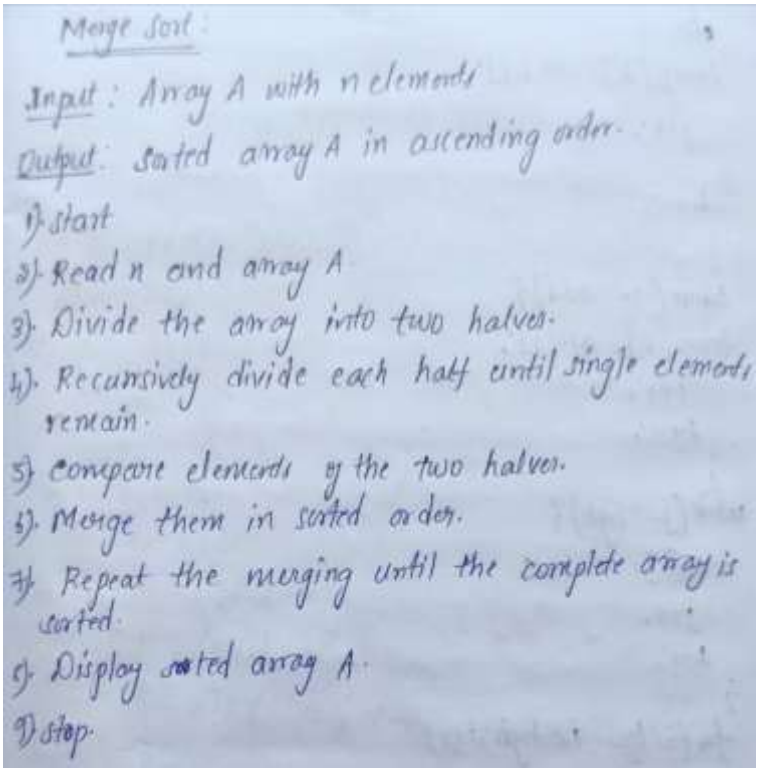
OUTPUT :


Enter array size: 4
Enter array elements: 5
6
2
3
Before sorting: 5 6 2 3
After sorting: 2 3 5 6
...Program finished with exit code 0
Press ENTER to exit console.

time complexity : Step Count / Frequency Count Method)



MERGE SORT ALGORITHM :



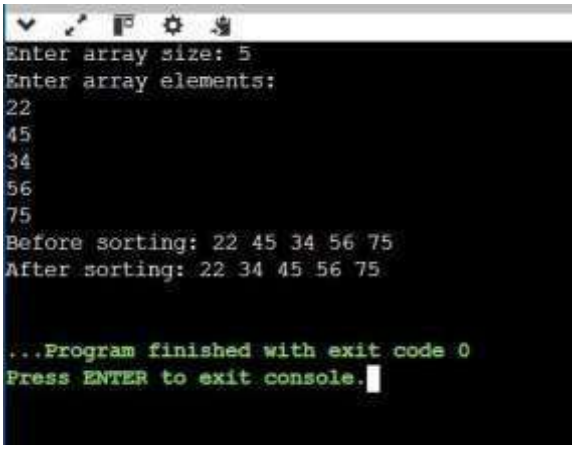
PROGRAM:

```
#include <iostream> using namespace
std;

class mergesort  { public:
   int size, arr[100], i;

   void arrayinput()       {          cout << "Enter array size:
";       cin >> size;        cout << "Enter array elements:\n";
for (i = 0; i < size; i++)        {              cin >> arr[i];
      }
  }
```

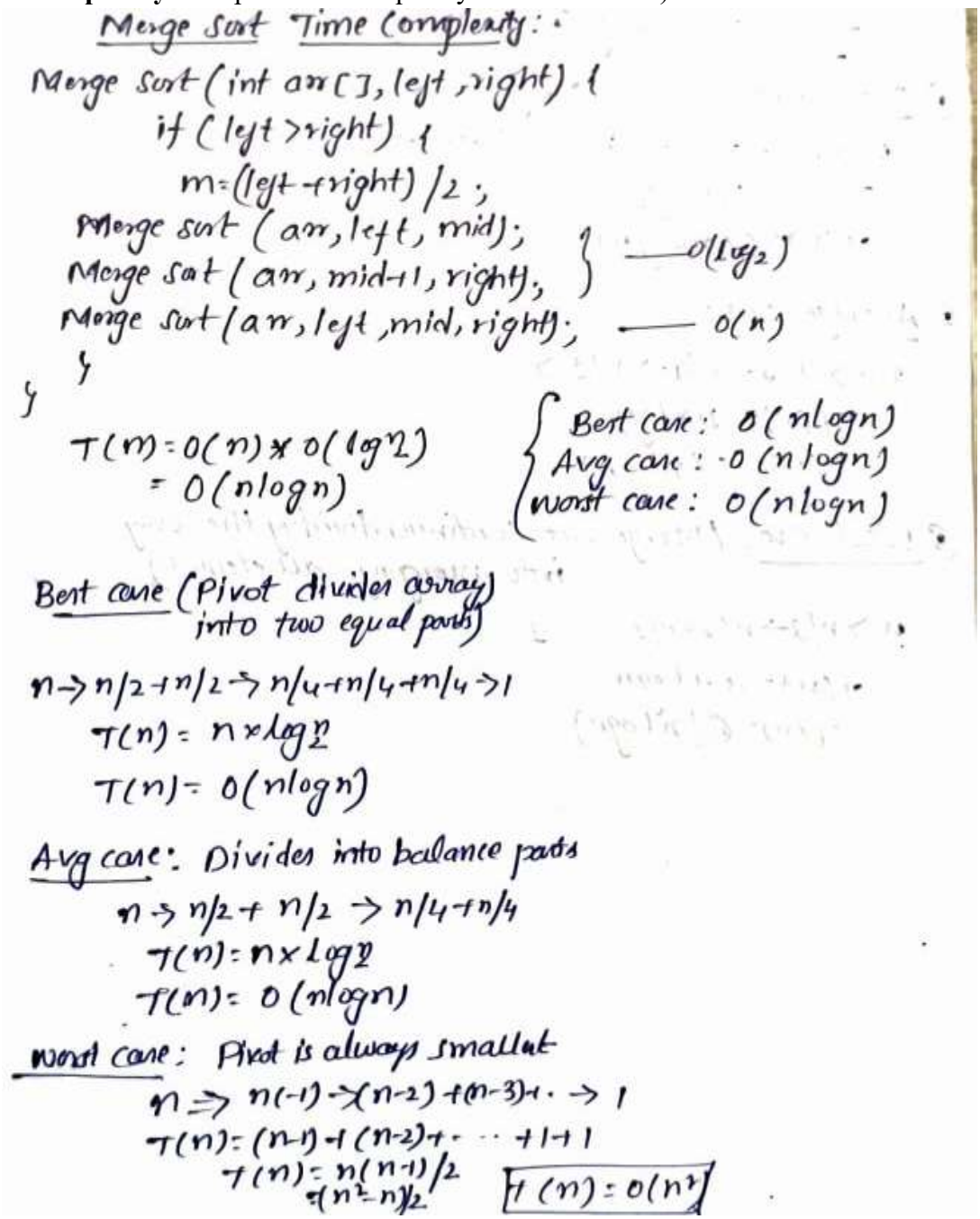
```
   void printarray()      {         for
(i = 0; i < size; i++)
{            cout << arr[i]
<< " ";
      }         cout
<< endl;
   }

   void merge(int low, int mid, int high)
{         int temp[100];         int i = low,
j = mid + 1, k = low;

      while (i <= mid && j <= high)
      {            if (arr[i]
<= arr[j])
         {
temp[k]     =     arr[i];
i++;           }           else
{              temp[k] = arr[j];
j++;
}           k++;

      }




      while (i <= mid)
      {            temp[k]
= arr[i];
i++;           k++;
      }

      while (j <= high)
      {             temp[k] = arr[j];           j++;           k++;
      }

      for (i = low; i <= high; i++)
      {            arr[i] = temp[i];
      }
   }
```

```
    void mergelogic(int low, int high)
    {          if (low < high)
        {              int mid = (low + high)
/ 2;

            mergelogic(low, mid);
mergelogic(mid + 1, high);

            merge(low, mid, high);
        }
    }

};

int main() {      mergesort
ms;

    ms.arrayinput();

    cout << "Before sorting: ";     ms.printarray();       ms.mergelogic(0, ms.size - 1);       cout << "After
sorting: ";     ms.printarray();

    return 0;  }
```

OUTPUT :



Time complexity : Step Count / Frequency Count Method)



Merge Sort Time Complexity :

```
Merge sort (int arr[], left, right) {
    if (left > right) {
        m = (left + right) / 2;
    Merge sort (arr, left, mid);      } —— O(log2)
    Merge sort (arr, mid+1, right);   }
    Merge sort (arr, left, mid, right); —— O(n)
    }
}
```

$T(n) = O(n) * O(\log_2 n)$
$= O(n \log n)$.

Best case: $O(n \log n)$
Avg case: $O(n \log n)$
worst case: $O(n \log n)$

Best case (Pivot divides array into two equal parts)

$n \to n/2 + n/2 \to n/4 + n/4 + n/4 \to 1$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

Avg case: Divides into ballance parts

$n \to n/2 + n/2 \to n/4 + n/4$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

worst case: Pivot is always smallest

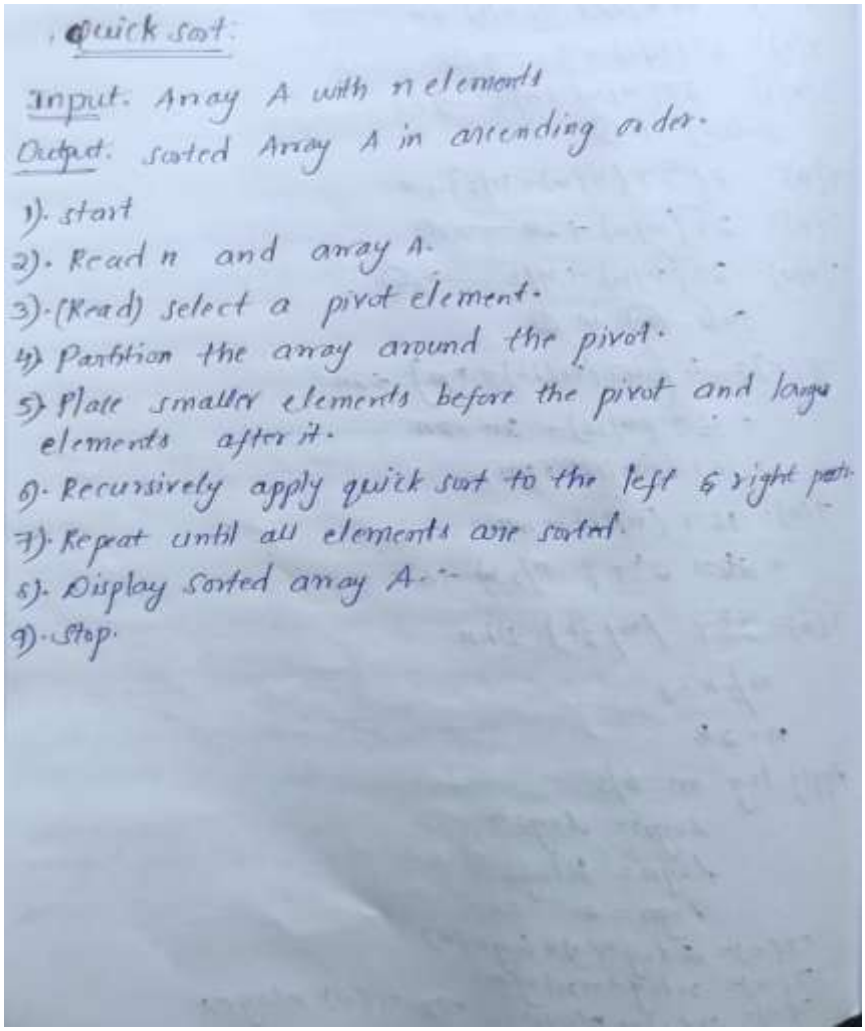
$n \Rightarrow n(-1) \to (n-2) + (n-3) + \cdots \to 1$

$T(n) = (n-1) + (n-2) + \cdots + 1 + 1$

$T(n) = n(n-1)/2$
$= (n^2 - n)/2$

$T(n) = O(n^2)$

QUICK SORT ALGORITHM:



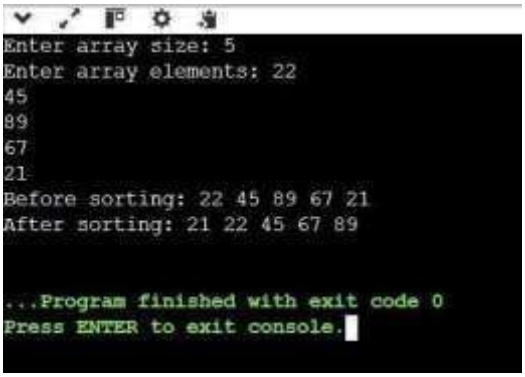
PROGRAM :

```
#include <iostream> #include
<utility>  using namespace
std;

class QuickSort  { public:
int size, arr[100];

   void arrayinput()       {         cout
<< "Enter array size: ";         cin >>
size;
```

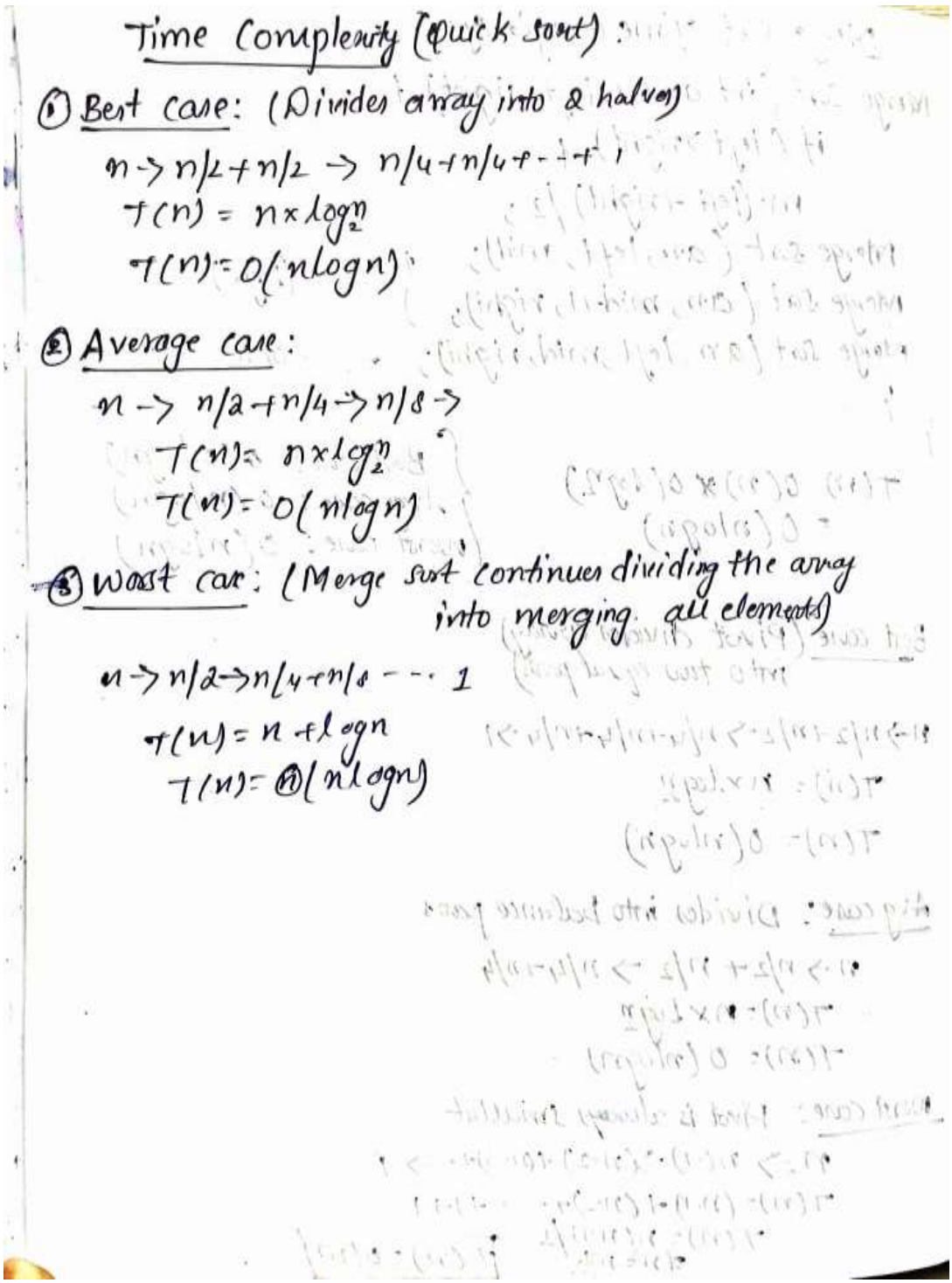
```
cout << "Enter array elements: ";

for (int i = 0; i < size; i++)

      {            cin
>> arr[i];
      }
   }

   void printarray()      {         for
(int i = 0; i < size; i++)
{            cout
<< arr[i] << " ";
}         cout
<< endl;
   }


   int partition(int lb, int ub)      {
int pivot = arr[lb];         int start
= lb;          int end
= ub;

      while (start < end)
      {            while (start <= ub && arr[start]
<= pivot)               start++;               while
(arr[end] > pivot)               end--;
if (start < end)               swap(arr[start],
arr[end]);
      }

      swap(arr[lb], arr[end]);
return end;
   }

   void quicksortlogic(int lb, int ub)
   {        if (lb < ub)        {            int
```

```
loc = partition(lb, ub);
quicksortlogic(lb, loc - 1);
quicksortlogic(loc + 1, ub);




   }          void sort()      {
quicksortlogic(0, size - 1);
   }
};

int main()
{

   QuickSort qs;

   qs.arrayinput();

   cout << "Before sorting: ";
qs.printarray();

qs.sort();

   cout << "After sorting: ";
qs.printarray();

   return 0;
}
```

OUTPUT :


Enter array size: 5
Enter array elements: 22
45
89
67
21
Before sorting: 22 45 89 67 21
After sorting: 21 22 45 67 89
...Program finished with exit code 0
Press ENTER to exit console.

TIME COMPLEXITY : Step Count / Frequency Count Method)



Time Complexity (Quick sort) :

① Best case: (Divides array into 2 halves)

$n \to n/2 + n/2 \to n/4 + n/4 + \cdots 1$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

② Average case:

$n \to n/2 + n/4 \to n/8 \to$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

③ Worst case: (Merge sort continues dividing the array into merging all elements)

$n \to n/2 \to n/4 + n/8 \cdots 1$

$T(n) = n + \log n$

$T(n) = \Theta(n \log n)$

Conclusion: